29.07.2024

SIMON MENNIG

Pull request reviews: How AI improves code quality and saves time

Coding & Frameworks, Software Quality Management

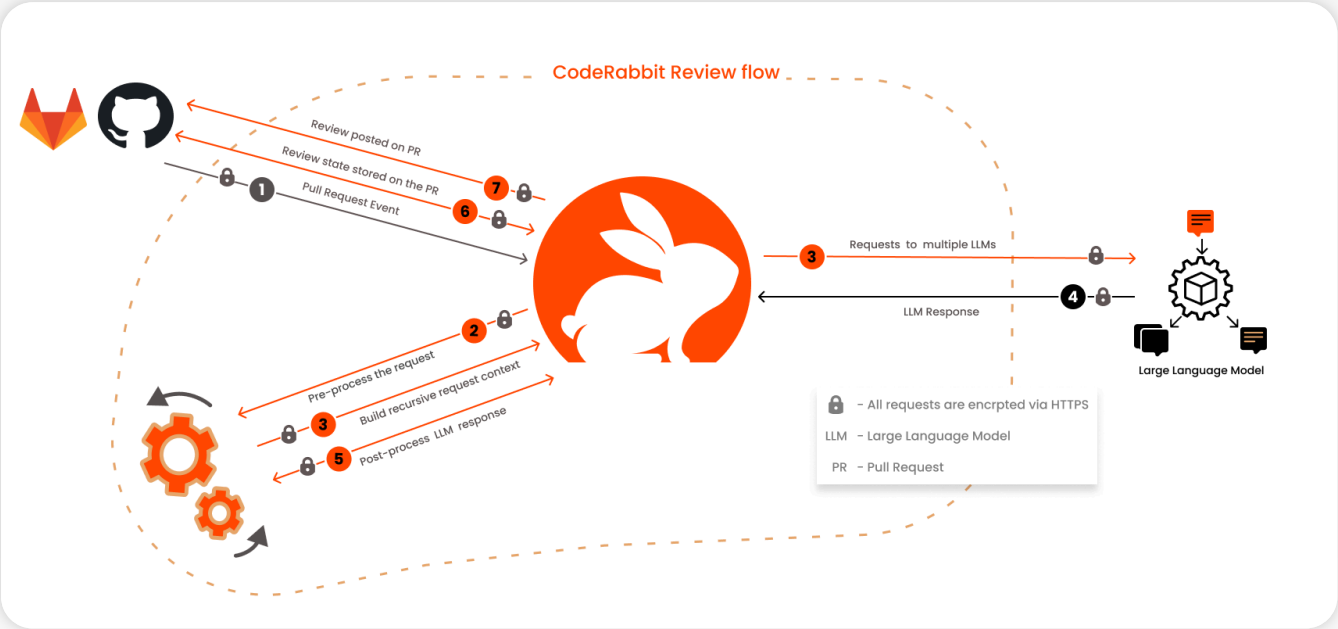
Inhalt der Seite

PR reviews are essential for ensuring code quality. However, they are often time-consuming and prone to errors. Tools such as GitHub Copilot already help with creating code - so why not review it too? LLMs can automate routine tasks and detect problems at an early stage. This not only improves quality, but also saves valuable time.

Available tools for AI-supported pull request reviews

For the public, there are solutions for GitHub or GitLab. Two of them are:

- CodeRabbit AIThis tool can only be used via the cloud.
- Codium AI PR Agent: This tool can be operated via the cloud or yourself (see Installation).



CodeRabbit AI Flow, source:[1]

Apart from the operating models, the functionality of the two tools is very similar. CodeRabbit presents it on its Website as follows.

Put simply, the pull request is read out and pre-processed. The pull request is then sent to one or more LLMs and evaluated. The result is then left as a comment in the pull request.

In the following, I will mainly focus on **Codium AI PR Agent** that it can be used more flexibly with the option of providing the LLM itself.

Example: Code reviews at KeepTime

Enough theory, using the time tracking tool KeepTime I have recreated two real pull requests.

Then I requested a review from Codium AI. The original pull requests are KeepTime #176 and KeepTime #173. The corresponding subsequent pull

DE EN

A review with Codium AI PR-Agent consists of three main parts: the "PR Reviewer Guide", the "PR Code Suggestions" and the "Changes Walkthrough".

Changes Walkthrough

This section provides an overview of the relevant files and the changes made. Here is an example: Changes walkthrough

	Relevant files
Enhancement	14
Tests	3
Dependencies	1
Configuration Changes	1

PR Reviewer Guide

The "PR Reviewer Guide" provides detailed guidance for the reviewer, including estimated effort and potential security concerns. A Example:

⏱️ **Estimated effort to review [1-5]**	4
? **Relevant tests**	yes
? **Security concerns**	**- Sensitive Information Exposure:** The application properties include sensitive data like username and password. Ensure these are securely managed, potentially using environment variables or a secure vault solution. - SQL Injection: Verify that all database queries, especially those constructed with parameters, are using prepared statements or

DE EN

⚡ **Key issues to review**	The `extractValue` method in `LoginController` assumes the format of the input string is always correct. It should handle cases where the format might not match expectations to avoid StringIndexOutOfBoundsException. **Code Duplication:** There is noticeable duplication in DTO classes for setting and getting properties. Consider using Lombok to reduce boilerplate code. **Error handling:** In `ProjectController` and `WorksController`, the exception handling is basic. It would be beneficial to add more specific error responses based on the caught exceptions to give clearer feedback to the client. **Validation:** Ensure that all necessary fields in DTOs are validated to prevent invalid data operations.

PR Code Suggestions

This part offers concrete suggestions for improving the code. Here is one Beispiel:

DE EN

Maintainability	parameterized tests to reduce duplication	10
	Refactor repeated code into a helper method	8
	Refactor not found response creation into a separate method for better maintainability	7
	Replace hardcoded property keys with static final strings	7
	Implement a Builder pattern for ProjectColorDTO to simplify object creation and improve code maintainability	6
	Refactor stream operations into a separate method to improve readability	6
Best practice	Rename test methods to be more descriptive	9
	Limit the scope of component scanning by specifying a base package	8
	Improve exception handling by using	8

DE EN

	Improve spacing for better readability	7
	Optimize data retrieval by using a specific query method instead of filtering in memory	9
	Use String.format for string concatenation to enhance readability and performance	6
Possible bug	Add null check for newValuedWorkDTO to prevent NullPointerException	9
	Correct file path handling to prevent FileNotFoundException	9
Enhancement	Add @Transactional annotation to ensure database consistency	8
	Use a constructor for initializing ColorDTO to enhance immutability and encapsulation	7

There are also code suggestions for most of the improvements, which can also be used directly.

Possible bug

Consider adding a null check for `inputStream` before calling `properties.load(inputStream)` to avoid potential `NullPointerException` if the resource is not found.

src/main/java/de/doubleslash/keeptime/view/SettingsController.java [655]

```
-properties.load(inputStream);
+if (inputStream != null) {
+    properties.load(inputStream);
+} else {
+    throw new FileNotFoundException("Property file '" + propertiesFilePath + "' not found
+}
```

☐ Apply this suggestion

▸ Suggestion importance[1-10]: 8

Source: Screenshot[2]

Conclusion

AI-powered tools such as CodeRabbit AI and Codium AI PR Agent provide valuable support for pull request reviews by helping developers focus on critical areas of code, uncover potential bugs and save time overall. These tools automate routine tasks and identify potential issues early on, which can improve the quality of code reviews. However, it is important to note that AI-based solutions may not be able to fully address current technical issues as they often lack specific context.

In future, for example, integration with an issue tracking tool such as Jira could provide the necessary context. This would allow not only technical aspects but also the technicality of changes to be checked.

Another point for possible further development would be the development of a dedicated user interface for these AI-supported tools. A dedicated UI could enable intuitive interaction.

Want to explore even more possible applications of AI technologies? Find out more and contact us here.

Learn more about software development

ABOUT ME

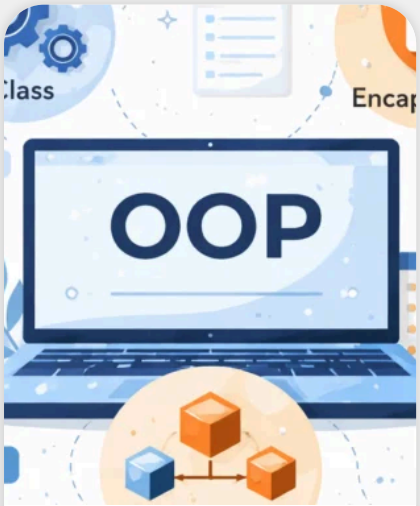
Simon Mennig

Simon Mennig studied Business Informatics (B.Sc.) at the Ravensburg-Weingarten University of Applied Sciences and Computer Science with a focus on Software Engineering (M.Sc.) at the HTWG in Constance. He has been working as a software developer at doubleSlash since 2020 and has expertise in the following areas, among others IoTcloud computing and Java.

All contributions from Simon Mennig

MORE ON THE TOPIC

Coding & Frameworks, Software Quality Management



Obsolete or simply misunderstood?

The professional „AI Coder“

LEARN MORE

Further information on our website and in our newsletter

Services & Products

Subscribe to newsletter

© doubleSlash Net-Business GmbH